Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL601**	**Course Name:**	**Data Analytics and Visualization Lab**

Name of Student:	**Pranjal Keshav Patil**
Roll No.:	**37**
Experiment No.:	**01**
Title of the Experiment:	**Getting introduced to data analytics libraries in Python and R.**
Date of Performance:	**06/01/2026**
Date of Submission:	**13/01/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms. Odilia Gonsalves

Signature:

Date:

Experiment No. 1

Aim: Introduction to Data analytics libraries in Python and R.

Objective- Understand the use of Python and R, to effectively use libraries for data science.

Description:

Why Choose Python?

Python is also a popular language for data science, and it comes with several advantages:

1. **Versatility**:
   Python is not just for data analysis. It is a general-purpose programming language, meaning it can be used for web development, automation, machine learning, and many other tasks. This versatility is beneficial for projects that require more than just data analysis.
2. **Ease of Learning**:
   Python has a simpler and much readable syntax compared to R, thus making it easier for beginners to learn and start coding quickly.
3. **Integration**:
   Python excels in integration. It works well with other programming languages, tools, and databases. For example, it can easily interface with web APIs, databases, and even big data platforms like Hadoop or Spark.
4. **Machine Learning**:
   Python is the leading language for machine learning and deep learning, with powerful libraries like scikit-learn and TensorFlow which make it easier to develop complex machine learning models.

Why Choose R?

1. **Specialized for Statistics**:
   R was built specifically for statistical analysis; thus, it has many tools and packages that make statistical tasks much easier.
2. **Visualization**:
   If you're working with data that needs to be visualized, R is particularly strong in this area. Libraries like ggplot2 allow you to create intricate and customized charts and plots that are both functional and visually appealing.

3. **Community Support**:
   The R community is huge and supportive, offering many resources like tutorials, forums, and libraries. If you encounter a problem, chances are someone else has already solved it.
4. **Reproducibility**:
   Reproducibility is critical in data science. With tools like Markdown and Shiny, you can create interactive reports and dashboards that others can easily reproduce.

R vs Python: Key Differences

Sr. No	Parameters	R	Python
1	Ease of Learning	A bit harder because its syntax is statistics-oriented	Easy and beginner-friendly because its syntax is simple and readable.
2	Statistical Analysis	Statistics-centric language (Built mainly for statistics)	Library-driven statistical support to do statistical work
3	Data Manipulation	R uses packages like *dplyr* to filter, sort, and transform data easily.	Python uses *Pandas*, which is extremely powerful and widely used for handling structured data like tables.
4	Visualization	R provides excellent visualization tools like *ggplot2*, which are highly customizable and widely used in research.	Python also supports visualization through libraries like *Matplotlib* and *Seaborn*, which are good but slightly less intuitive than R for complex statistical plots.

5	Machine Learning	R supports machine learning but has limited tools compared to Python.	Python is the leader in machine learning with libraries like scikit-learn, TensorFlow, and PyTorch, making it ideal for AI and advanced analytics.
6	Community Support	R has a strong community of researchers, statisticians, and academics.	Python has a huge global community including developers, companies, and researchers.

7	Integration	R has limited integration with other technologies like APIs and web apps.	Excellent integration with APIs, databases, and big data tools
8	Reproducibility	R provides excellent tools like *R Markdown* to create reproducible reports with code and output together.	Moderate (Python supports reproducibility (e.g., Jupyter notebooks), but it is not as structured as R.)
9	Performance	R can be slower when handling very large applications or systems.	Python is faster and more scalable, especially when combined with optimized libraries.
10	Versatility	R is mainly used for data analysis, statistics, and research.	More versatile (Python is a general-purpose language used for web development, AI, automation, apps, and more.)

R packages:

Sr No	Name of Package	Use/Description	Functions and its description
1	hunspell	**hunspell** is an R package used for **spell checking, word analysis, and text processing**.(NLP) It is based on the Hunspell library (used in software like LibreOffice and browsers).	hunspell() - Checks the spelling of words or text strings. hunspell("Ths is a smple txt") Output: Ths, smple, txt
			hunspell_suggest() - Provides suggested correct spellings for misspelled words. hunspell_suggest("smple") Output: "simple", "sample"
			hunspell_check() - Returns TRUE or FALSE • TRUE = correct spelling • FALSE = incorrect hunspell_check("hello") TRUE
			hunspell_analyze() - Analyzes words to show roots, prefixes, and suffixes. hunspell_analyze("unhappiness") Output (simple meaning): Prefix: un- Root: happy Suffix: -ness
			hunspell_stem() - Extracts the base or root form of words. hunspell_stem("playing") "play"
2	magick	Enables image reading, editing, transformation, with support for multiple formats.	image_read() - Reads an image file into R. (Loads an image into R) img <- image_read("photo.jpg")
			image_resize() - Changes the size of an image. image_resize(img, "300x300")

		Useful for image processing and manipulation. It is based on the powerful ImageMagick library.	image_write() - Saves the edited image image_write(img, "output.png")
			image_rotate() - Rotates an image by a specified angle. image_rotate(img, 90)
			image_convert() - Converts an image to a different format. image_convert(img, format = "png")
3	profvis	**profvis** is an R package used to **analyze the performance of your code**. It shows **which parts of your code are slow or using more memory**.	profvis() - Shows a visual report of execution time , Main function to profile code profvis({ # your code here })
			print.profvis() - Prints profiling results in a readable format.
			pause() - Temporarily pauses profiling during execution.
			resume() - Resumes profiling after a pause.
			is_paused() - Checks whether profiling is currently paused.
4	fs	Simple and reliable way to manage files and directories, allowing users to create, read, move, and delete them efficiently. Used to **work with files and folders (directories)** easily.	file_create() - Creates a new empty file. file_create("test.txt")
			file_delete() - Deletes a specified file. file_delete("test.txt")
			dir_create() - Creates a new directory / folder dir_create("my_folder")
			dir_delete() - Deletes a specified directory.
			dir_ls() - Lists all files and folders in a directory. dir_ls("my_folder")

5	tidytext	Used for text mining and NLP, converting text into a tidy format (clean and structured), for easy analysis, manipulation, and visualization.	unnest_tokens() - Splits text into words, sentences, or tokens unnest_tokens(data, word, text)
			summarize() - Summarizes text statistics, like word counts or sentiment scores.
			count() - Counts the frequency of words or tokens. count(word)
			filter() - Filters words or tokens based on conditions. (Removes unwanted words)
			group_by() - Groups text data for aggregated analysis.

Python packages:

Sr No	Name of Package	Use/Description	Functions and its description
1	NumPy	**NumPy (Numerical Python)** is a Python library used for **fast numerical calculations and working with arrays (lists of numbers)** for data analysis, machine learning, and scientific calculations.	array() - Creates a NumPy array from a list or tuple. import numpy as np arr = np.array([1, 2, 3])
			mean() - Calculates the mean (average) of array elements. np.mean(arr)
			sum() - Returns the sum of array elements. np.sum(arr)
			zeros() - Creates an array filled with zeros. np.zeros((2,3))

			ones() - Creates an array filled with ones. np.ones((2,3))
2	Pandas	Data analysis and manipulation using DataFrame and Series to clean, filter, and explore data efficiently. It helps you work with **tables (like Excel sheets)** in Python.	pd.DataFrame() - Creates a DataFrame(tables) from lists, dictionaries, or arrays. import pandas as pd data = pd.DataFrame({"Name": ["A", "B"], "Age": [20, 25]})
			pd.Series() - Creates a one-dimensional labeled array (Series). (Single Column) pd.Series([1, 2, 3])
			head() - Shows the first few rows of a DataFrame data.head()
			tail() - Shows the last few rows of a DataFrame.
			sort_values() - Sorts data data.sort_values("Age")
3	Matplotlib	Plotting graphs and visualizing data easily. import matplotlib.pyplot as plt	plt.plot() - Plots a line graph.
			plt.bar() - Creates a bar chart.
			plt.pie() - Draws a pie chart.

			plt.title() - Adds a title to the plot.
			plt.show() - Displays the plot.

4	scikit-learn	For machine learning, used for building and evaluating predictive models like classification, regression, and clustering. It helps you build models that can **learn from data and make predictions**.	train_test_split() - Splits data into training and testing sets. from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(X, y)
			accuracy_score() – Measures the accuracy of a classification model by comparing predicted values with actual values. from sklearn.metrics import accuracy_score accuracy_score(y_test, y_pred)
			LinearRegression() - Creates and fits a linear regression model. Used for predicting numerical values from sklearn.linear_model import LinearRegression model = LinearRegression() model.fit(X_train, y_train)

			LogisticRegression() - Creates and fits a logistic regression model for classification. Used for classification (yes/no type problems)
			predict() - Predicts outcomes using the trained model. model.predict(X_test)
5	Flask	Python framework for building web applications. Allows developers to create websites, APIs, and backend services quickly.	run() - Starts the Flask web server. app.run(debug=True)
			Flask() - Creates a Flask application from flask import Flask app = Flask(__name__)
			request - Accesses data sent in HTTP requests (GET, POST).
			route() - Defines URL routes and links them to Python functions. @app.route("/") def home(): return "Hello World"
			redirect() - Redirects the user to a different URL.

Common IDEs

RStudio
Jupyter Notebook
Spyder
PyCharm
VS Code

Conclusion-

Python and R, with their powerful libraries, provide essential tools for data analytics, enabling efficient data manipulation, visualization, statistical analysis, and machine learning.

- Python: Versatile and beginner-friendly, ideal for data analysis, machine learning, and integration with other tools.
- R: Specialized for statistical analysis and data visualization, with strong support for research and reproducible workflows.